

If we find an element $A[j]$ at array index 'j' such that ($j < i$) and $A[j] > A[i]$, then $A[j]$ is the PGE for $A[i]$ and PGE index for 'i' is 'j'. If we hit the beginning of the array without finding PGE, then that particular array element has no PGE in the array, so we can mark its PGE index as -1. What is the time complexity of this brute force approach? Since we scan backwards the entire array to the left of the current element at each index 'i', the time complexity is $O(N^2)$, where N is the length of the array.

We can improve on this $O(N^2)$ complexity by sorting the array first and remembering its original index position in the unsorted array to calculate the PGE. This would give us a time complexity of $O(\log n)$. Again, we are making an unnecessary effort by sorting the entire array, whereas all we need is the location of the PGE for each element. How can we reduce the time complexity to $O(N)$?

We can use a stack to track the PGE for each index. In this case, we push the first array element on to the stack. Now, as we move right on the array, we compare each element to the stack top. As long as the stack top is smaller or equal to the current element, we have not found the PGE for the latter. Thus, we can pop the topmost element from the stack.

We will then continue popping elements until we find the stack top element greater than the current element – at which point we have found its PGE. We then record the PGE for the current element and push it on to the stack (since the current element can be PGE for the new incoming array elements and hence needs to be against future elements). An element can be discarded from the stack (popped off) when it is smaller than the current element because it can't be the PGE for any future elements (since the current element is greater than it). I will leave it to our readers to write out the complete code.

How do we know that this method will give us $O(N)$ complexity? At first glance, we seem to be comparing each element on the stack with the current element, until we hit the PGE (which seems pretty similar to our brute force approach). The key point is that each array element gets pushed onto the stack and gets popped from the stack only once. And once it gets popped, it can no longer be compared against. We compare only against the stack top, the current element.

If the stack top is lesser, it will get popped from the stack and is no longer involved in any future comparisons. If it is greater, we know that it is the PGE for the current element, and the latter gets pushed onto the stack and becomes the new stack top. The maximum depth the stack can grow is $O(N)$ and hence time complexity is $O(N)$.

As we discuss and solve coding interview questions, we need to build the ability to recognise the common patterns underlying seemingly different problems. This would allow us to solve multiple problems with a single common pattern wherever it is applicable. Now let us discuss a few more coding questions.

1. You are given a binary tree of N nodes. How will you implement a level order traversal of this binary tree?
2. You are asked to implement a stack using two queues. How will you support standard stack operations in this case? Instead of two queues, can you implement a stack using one queue only as the underlying data structure?
3. You are given a non-decreasing array of N integer elements and an integer K. The task is to remove K elements from the array such that the maximum difference between adjacent elements is minimum.
4. You are given an array A of N integer elements and an integer value K. For each and every contiguous sub-array of size K of the original array A, find the maximum for that sub-array.
5. You are given an array of N integers with some of the elements being duplicates. The array values can range between 1 to N. You need to find the number of sub-arrays which start and end with the same element.
6. You are given an array containing two halves such that each half is sorted. Your task is to create a sorted array in place. What is the time and space complexity?
7. You are given a graph G represented by its vertices V and edges E. Given a pair of vertices (u,v), find all the paths between these two vertices in graph G.
8. You are given a set S of N integers. You are asked to find the power set of S. First assume that S does not contain any duplicates and derive your solution. If S contains duplicates, your resulting power set should not contain duplicate sub-sets. How do you find a solution for this case?
9. You are given a sentence S in which all white spaces between words have been removed so that words are NOT separated. For example, you are given a sentence $S = \text{"whatisthecapitalofIndia"}$. You are also given a dictionary containing all valid words. Can you create a valid word list for the sentence using the dictionary? This is basically the tokenisation problem in natural language processing for languages where words need not be separated by white spaces. What is the time and space complexity of your solution?
10. Given two strings S1 and S2, find how many times S2 can occur as a sub-string in S1.

Feel free to reach out to me over LinkedIn/email if you need any help in your coding interview preparation. If you have any favourite programming questions/software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyasm@yahoo.com.

Wishing all our readers happy coding until next month! Stay healthy and safe. 

 **By: Sandya Mannarswamy**

The author is an expert in natural language processing and is currently working as an independent researcher. Her interests include natural language processing, machine learning and AI.